IMPROVED SOFTWARE REQUIREMENTS SPECIFICATION

1) INTRODUCTION

1.1) PURPOSE:

The **Noughts & Crosses (Tic-Tac-Toe)** game is designed to simulate a two-player game on a 3x3 grid, with one player being controlled by the user and the other by a robot (SwiftBot). The project will be implemented using a command-line interface (CLI), with the game logic running on a computer and the SwiftBot executing physical movements on the board based on the moves chosen by the player or the robot itself.

1.2) SCOPE:

a) **Game Logic:** Handling player moves, win conditions, and game state.
b) **User Interaction:** Providing an interface for the user to input their moves, view the board state, and interact with the system through the CLI.
c) **SwiftBot Integration**: Enabling the robot (SwiftBot) to make moves, respond to user interactions, and show physical feedback (e.g., blinking LEDs for win/loss/draw).
d) **Logging and Persistence:** Recording game results in a persistent log and maintaining a scoreboard to track performance over multiple rounds.
e) **Calibration and Testing:** Ensuring that the SwiftBot's movements are accurate within a defined tolerance and calibrating the robot to fit the physical board's grid.

1.3) ACTORS:

a) **User:** The person playing the game. They will enter their name, select moves on the 3x3 grid, and interact with the system through the CLI. The User controls the first player, and their actions determine the state of the game, including input validation and move confirmation.
b) **SwiftBot (Robot):** A robot controlled by the system that makes random moves on the 3x3 grid when it is the second player's turn. The SwiftBot will move physically on the board, blink LEDs to indicate the state of the game (win/loss/draw) and return to its starting position after each move. It also performs basic calibration to ensure its movements are accurate.
c) **System:** The software that controls the game logic, validates user input, updates the game board, maintains the scoreboard, and handles all aspects of the game from start to finish. The system includes the CLI interface, game state management, and the logic for interacting with the SwiftBot.

1.4) DEFINITIONS, ACRONYMS, AND ABBREVIATION:

a) **SRS:** Software Requirements Specification
b) **UI:** User Interface
c) **CLI:** Command-Line Interface

1.5) SRS OVERVIEW:

This SRS outlines the functional and non-functional requirements for noughts & crosses, as well as outlining any additional functionalities that should be integrated alongside the required ones.

2) FUNCTIONAL REQUIREMENTS

2.1) PHYSICAL & ASCII GAMEBOARD DETAILS & GAME PIECES:

a) a physical board for the swiftbot to move on
b) The physical board will be a 3x3 (9 squares), an area of 75 by 75cm and will be numbered [1-3]. Each square will measure 25x25 cm.
c) each player will have 4 game pieces each: X or O, to mark their chosen position.
d) There will be a command-line ASCII representation of the 3x3 board digitally for the user to execute their moves, with clear row/column labels mapping out the coordinates of moves.
e) All inputs and any outputs should be handled by the command-line interface.

2.2) ALIGNING BUTTONS WITH CORRESPONDING LED COLOURS:

a) Button A (front left): Red
b) Button B (back left): Green
c) Button X (front right): Blue
d) Button Y (back right): Yellow

2.3) DISPLAY WELCOME SCREEN & USER REGISTRATION:

a) The program will need to display a welcome screen with a user-readable message. It should have a brief description of the game rules, which must also be clear.
b) The program shall begin the registration sequence when the user presses the SwiftBot 'A' button.
c) the program should prompt the user to enter a username via the keyboard. The name should be in a string format; any other formats is invalid.
d) a name will be randomly generated and assigned to the swiftbot after the user enters a username which can be configured manually.

2.4) GENERATE & ROLL DICE:

a) The program will roll a virtual 6-sided die for both the player and swiftbot automatically.
b) The player with the highest dice roll goes first and will be assigned the 'O' piece, whilst the other will have the 'X' piece.
c) if there's a tie, the system shall re-roll until a single higher value is produced and the starter is determined.

2.5) USER'S TURN:

a) For the user's turn, the System shall prompt the User to enter a move in the format [row, column] within the range 1-3 in full integers.
b) The System shall validate and verify that the user input is numeric and in the right format.
c) If invalid, the System shall show an error and re-prompt.

d) The System shall check whether the selected square is empty
e) if a square is an unavailable, the System shall notify the User and re-prompt for a different move.
f) When the User provides a valid move, the System shall mark the internal board state with the User's piece ('O' or 'X') on the square.
g) The System shall ensure that the user cannot make consecutive moves
h) update the displayed ASCII board after the move is successful
i) displays the following message: **'[User – O/X] moved to square [r, c]'** after each successful move

2.6) SWIFTBOT'S TURN:

a) When it's the swiftbot's turn, the robot will randomly select its move to its target square within the range
b) display the chosen square to the User *before* instructing the SwiftBot to move.
c) After going to the chosen square, the swiftbot will then execute a blink sequence in green 3 times upon arriving to the target square.
d) The swiftbot will then return to its starting position in the board after the move.
e) the system should then display the move summary for swiftbot move: **[SwiftbotName – Piece] moved to square [r, c]]**

2.7) WIN/DRAW CONDITIONS, TRACING WINNING LINE & BLINKS LEDs:

a) If the user wins, the swiftbot will trace the winning line and blinks green 3 times before and after tracing for an 'O' win.
b) If the swiftbot wins, the swiftbot will trace the winning line and blinks red 3 times before and after tracing for an 'X' win.
c) If it's a draw, the swiftbot will spin once and blink in blue 3 times before and after the move.

2.8) MAINTAINING LOG FILE & UPDATE BOARD:

a) After each round (win or draw), the System shall append the following information in the log file: round number, the playing piece (i.e., 'O' or 'X'), the move positions (row and column), game result (win/draw), the winner player name or draw and current date & time.
b) The System shall update and maintain the scoreboard across rounds to track user performance
c) display the scoreboard results after each round.
d) When the user quits, the System shall close the log file and terminate the program
e) When the user or swiftbot wins a round, they will be awarded a score

2.9) PLAY AGAIN/QUIT OPTIONS:

a) **Button Y:** play another round and the system will reset the board and start a new game
b) **Button X:** quit and terminate the program

3) NON-FUNCTIONAL REQUIREMENTS

3.1) USABILITY:

a) The game interface will be implemented as a simple command line interface console that is easy to navigate and intuitive.
b) Console UI formatting and design structure should be clear, simple and organised, displaying clear information.
c) Error messages should be clear, concise and on to the point. Non-technical users should not see that confusing paragraph/block of error message.
d) Displays game board in console game in clear and precise format.

3.2) PERFORMANCE

a) The system response time should be shorter from 1-2 seconds for updating player moves, calculating positions, updating boards.

3.3) RELIABILITY

a) There will be effective and reliable input validation to check the format, data type and numeric ranges for user input.
b) Efficient error-handling to manage exceptions, prevent unexpected outcomes like crashes and bugs and handle it gracefully.

3.4) PORTABILITY

a) The program should run on multiple platforms that is java-compatible and supports java runtime environment

3.5) MAINTAINABILITY

a) Code comments should be provided to explain each method's functionalities and purpose.
b) Code should be modular, which each block of code representing a specialised method performing a single function

3.6) SCALABILITY

a) The system should only have two players – the user and the swiftbot, as this is a two-player game.
b) A log file that consistently tracks and records game events and actions like player names, scoring, player positions, movements, wins, losses and draws.

3.7) SECURITY

a) the game doesn't handle sensitive data and therefore does not need any security strategies and backups to handle such.

3.8) USER INTERFACE

a) All inputs and outputs will be handled via the command-line console window
b) The user will interact with the system via the swiftbot buttons and the physical keyboard.
c) The system will handle swiftbot movements during each round
d) Swiftbot LEDs will light up depending on the button pressed, for example, pressing button 'A' will activate the red LED.
e) The user is expected to input their moves in an integer format and should be within the appropriate range of the board.

4) ADDITIONAL FUNCTIONALITIES

4.1) Choice to undo the last move the player makes but has a limit of 2 times (including the players first move) to prevent them from too easily gaining an advantage and allow them to think quicker and more strategically to balance out the odds of the game.

4.2) a timer will be set where the user will have to put their piece on the game board and will allow the player to adjust from 5-10 seconds the time the program will recognise the validity of the move. After that time, the move will become invalid and will give that turn to the next player.

4.3) The game will have a difficulty settings option at the start of the game, in the introduction, where the command line interface will display options, which are 'easy', 'medium' and 'hard' after the introduction of the game, asking the user '*what game mode? Enter easy, medium or hard*'. the difficulty of the game will depend on the two functionalities stated above:

a) How much '**undo's**' the user will have depending on game difficulty. For instance, the 'easy' mode will have 2 new move undo's, meaning the user can revise their move choices back by removing their piece twice. Whereas the medium and hard modes will have 1 and none respectively.

b) This will prompt the program to ask, '**do you want to undo your last move, enter yes or no?**', if yes, the gameboard will remove your last piece to start again or if no, then the pieces stays there.

c) The program will keep track of the number of times the user uses the undo option based on their difficulty. This value will be enclosed in brackets like '***Undo move chances: 2***' after the program asks yes or no. this will be shown in the command line as ***'do you want to undo your last move, enter yes or no? (Undo move chances: 2)'***,

d) For each turn, a timer countdown will be enforced as a time limit the user is allowed during their move. For the easy mode, there won't be a timer but a time limit of 10 seconds for the medium mode and 5 seconds for the hard mode will be applied.